CPE 233

Syllabus Review/Course Overview
Register Review
Memory Introduction
Experiment #1 Introduction

3:8 Decoder

SystemVerilog

```
module decoder3_8(input  logic [2:0] a,
                  output logic [7:0] y);
  always_comb
    case(a)
      3'b000:  y=8'b00000001;
      3'b001:  y=8'b00000010;
      3'b010:  y=8'b00000100;
      3'b011:  y=8'b00001000;
      3'b100:  y=8'b00010000;
      3'b101:  y=8'b00100000;
      3'b110:  y=8'b01000000;
      3'b111:  y=8'b10000000;
      default: y=8'bxxxxxxxx;
    endcase
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity decoder3_8 is
  port(a: in  STD_LOGIC_VECTOR(2 downto 0);
       y: out STD_LOGIC_VECTOR(7 downto 0));
end;

architecture synth of decoder3_8 is
begin
  process(all) begin
    case a is
      when "000" =>  y <= "00000001";
      when "001" =>  y <= "00000010";
      when "010" =>  y <= "00000100";
      when "011" =>  y <= "00001000";
      when "100" =>  y <= "00010000";
      when "101" =>  y <= "00100000";
      when "110" =>  y <= "01000000";
      when "111" =>  y <= "10000000";
      when others => y <= "XXXXXXXX";
    end case;
  end process;
end;
```

Parameterized N-Bit 2:1 Multiplexers

SystemVerilog

```
module mux2
  #(parameter width=8)
   (input  logic [width-1:0] d0, d1,
    input  logic             s,
    output logic [width-1:0] y);

   assign y=s ? d1 : d0;
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity mux2 is
  generic(width: integer := 8);
  port(d0,
    d1: in  STD_LOGIC_VECTOR(width-1 downto 0);
    s:  in  STD_LOGIC;
    y:  out STD_LOGIC_VECTOR(width-1 downto 0));
end;

architecture synth of mux2 is
begin
  y <= d1 when s else d0;
end;
```

Structural model of 4:1 Mux

SystemVerilog

```
module mux4_8(input  logic [7:0] d0, d1, d2, d3,
              input  logic [1:0] s,
              output logic [7:0] y);

  logic [7:0] low, hi;

  mux2 lowmux(d0, d1, s[0], low);
  mux2 himux(d2, d3, s[0], hi);
  mux2 outmux(low, hi, s[1], y);
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity mux4_8 is
  port(d0, d1, d2,
       d3: in  STD_LOGIC_VECTOR(7 downto 0);
        s: in  STD_LOGIC_VECTOR(1 downto 0);
        y: out STD_LOGIC_VECTOR(7 downto 0));
end;

architecture struct of mux4_8 is
  component mux2
    generic(width: integer := 8);
    port(d0,
         d1: in  STD_LOGIC_VECTOR(width-1 downto
         s:  in  STD_LOGIC;
         y:  out STD_LOGIC_VECTOR(width-1 downto
  end component;
  signal low, hi: STD_LOGIC_VECTOR(7 downto 0);
begin
  lowmux: mux2 port map(d0, d1, s(0), low);
  himux:  mux2 port map(d2, d3, s(0), hi);
  outmux: mux2 port map(low, hi, s(1), y);
end;
```

Adder

SystemVerilog

```
module adder #(parameter N = 8)
              (input  logic [N-1:0] a, b,
               input  logic         cin,
               output logic [N-1:0] s,
               output logic         cout);

  assign {cout, s} = a + b + cin;
endmodule
```

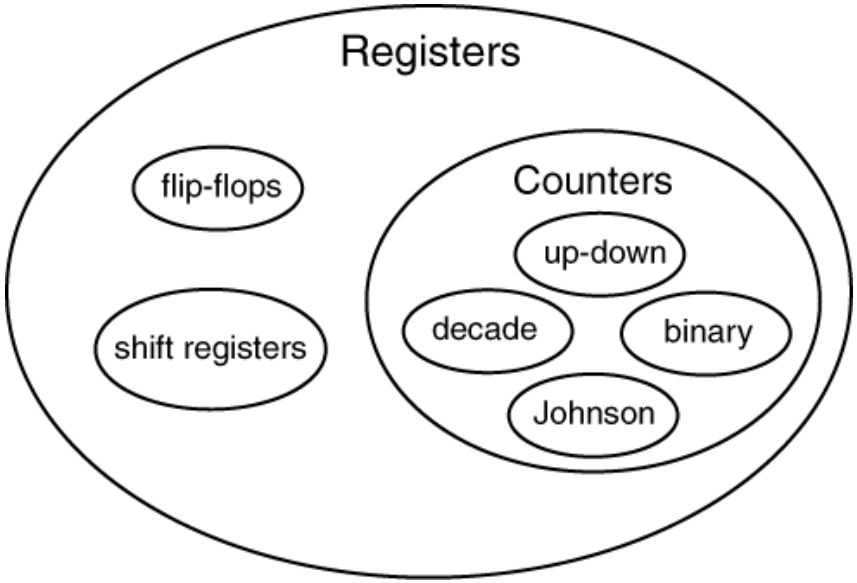
VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD_UNSIGNED.ALL;

entity adder is
   generic(N: integer := 8);
   port(a, b: in  STD_LOGIC_VECTOR(N-1 downto 0);
        cin:  in  STD_LOGIC;
        s:    out STD_LOGIC_VECTOR(N-1 downto 0);
        cout: out STD_LOGIC);
end;

architecture synth of adder is
  signal result: STD_LOGIC_VECTOR(N downto 0);
begin
  result <= ("0" & a) + ("0" & b) + cin;
  s      <= result(N-1 downto 0);
  cout   <= result(N);
end;
```

Registers

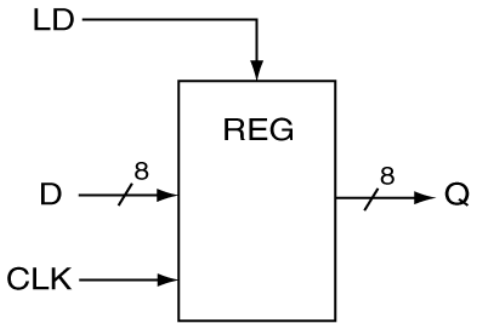
- Review
  - Registers: n flip-flops
  - Flop-flops: 1-bit registers

- Register Types
  - Flip-flops
  - Counters
  - Shift register



Registers with Load Control

- Registers can have many inputs
  - Parallel loads (enables register to latch new value)



Top-level block diagram

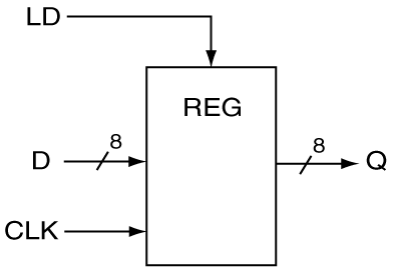
```
module flop(input logic clk,
            input logic [3:0] d,
            output logic [3:0] q);
    always_ff @(posedge clk)
          q <= d;
endmodule
```

SystemVerilog

```
entity reg is
    Port ( D : in std_logic_vector(7 downto 0);
           CLK,LD : in std_logic;
           Q : out std_logic_vector(7 downto 0));
end reg;

architecture my_reg of reg is
begin
   process (D, CLK, LD)
   begin
      if (rising_edge(CLK)) then
         if (LD = '1') then
            Q <= D;
         end if;
      end if;
   end process;
end my_reg;
```

VHDL

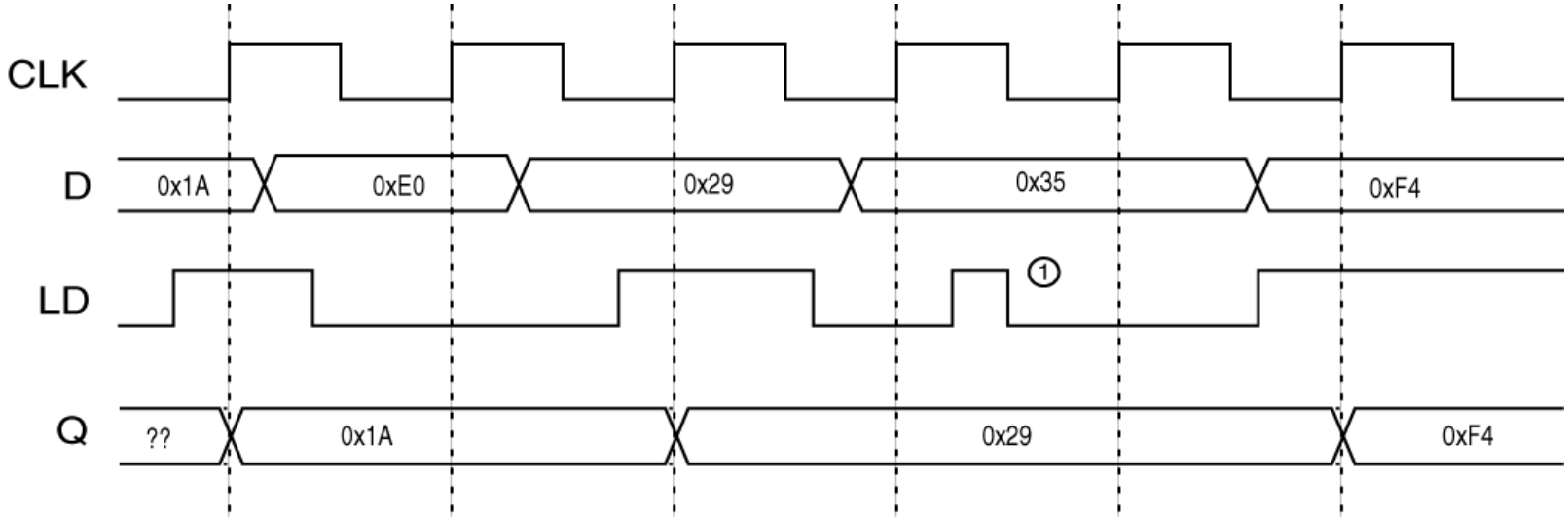
Registers with Load Controls



Top-level block diagram

```
architecture my_reg of reg is
begin
   process (D,CLK)
   begin
      if (rising_edge(CLK)) then
         if (LD = '1') then
            Q <= D;
         end if;
      end if;
   end process;
end my_reg;
```

VHDL Model



Sample timing diagram

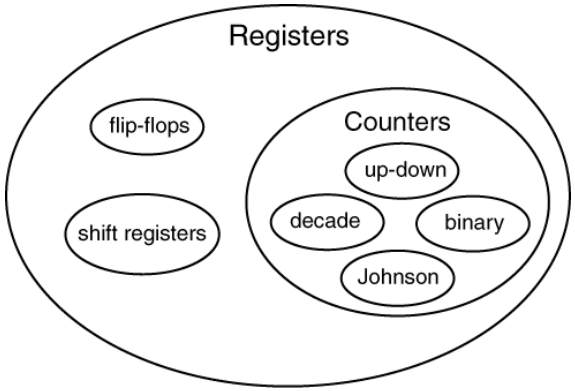
Signal-Based Register

- Registers can be “inline”
  - An 8-bit register

```
architecture my_ckt of my_ckt is
   signal r_reg : std_logic_vector(7 downto 0);
begin

   process (r_reg,CLK)
   begin
      if (rising_edge(CLK)) then
         r_reg <= some_value;
      end if;
   end process;


  ; more junk

end my_reg;
```

VHDL Model

Counters



- Special type of register
  - Outputs a known & repeatable sequence
  - Sequence == the “count”
- Most count operations are synchronous
- Types of counters
  - Decade counter (count in base 10)
  - Binary counter (count in base 2)
  - Twisted ring (special sequence)

Counter Information

- Up, Down, Up/Down Counters
- Typical synchronous operations
  - Increment, decrement
- Typical asynchronous operations
  - Load, Clear, Preset

- Counter Lingo
  - Overflow/underflow: (max→0 / 0→max)
  - Circular: sequence repeats automatically
  - Count enable: control signal for incr/decr
  - Ripple Carry Out (RCO): output at end of count sequence

Modeling Counters

- Many ways to model them
  - FSM
    - Generally for specialty counter
  - Verilog
    - Start from template
- Suggested approach
  - Use VHDL counter template as starting point to model counter to fit your needs

Counter

SystemVerilog

```
module counter #(parameter N = 8)
                (input  logic clk,
                 input  logic reset,
                 output logic [N-1:0] q);

  always_ff @(posedge clk, posedge reset)
    if (reset) q <= 0;
    else       q <= q + 1;
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD_UNSIGNED.ALL;

entity counter is
  generic(N: integer := 8);
  port(clk, reset: in  STD_LOGIC;
       q:          out STD_LOGIC_VECTOR(N-1 downto 0));
end;

architecture synth of counter is
begin
  process(clk, reset) begin
    if reset then                q <= (OTHERS => '0');
    elsif rising_edge(clk) then q <= q + '1';
    end if;
  end process;
end;
```

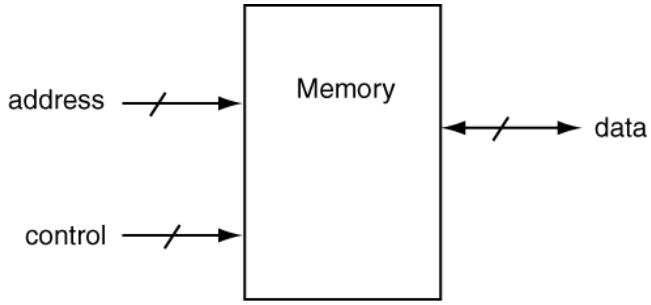
Memory

- The truth about memory
  - Many different types out there

Memory

- Basic operation: Memory stores bits
- Memory signals
  1. Data (input and/or output)
  2. Address
  3. Control

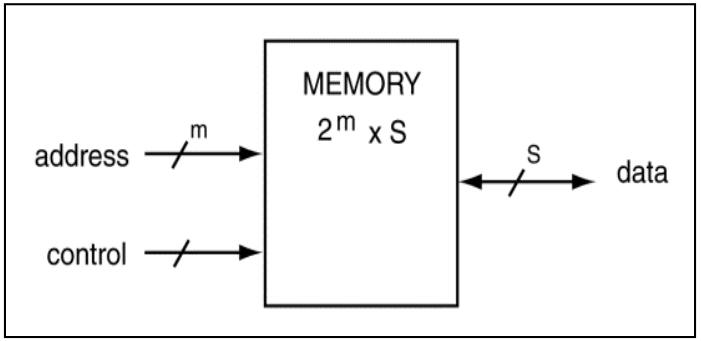
- Memory operations
  - Read
    - Copy some in memory to outside world
      - Does not change memory
  - Write
    - Put new bits in memory
      - Changes memory



Generic Memory Module

Memory Operations & Lingo

- *Read*: retrieve data from device
- *Write*: store data in device
- Capacity: how much data the memory stores



Memory Module

- Data store in groups of bits: *words*
- Smallest accessible group of bits = *s (bit width, word length)*
- Number of address lines = *m*
- Memory capacity in words = 2^m
- Memory capacity in bits = 2^m *x s*

Memory

- *There are many types of memory out there but they are typically classified as two types*

- Three main types of memory
  - ROM: read only memory (EPROM, PROM, Masked ROM)
  - RAM: random access memory (DRAM, SRAM)
  - Hybrid: flash

- Definitions don’t have much literal meaning
  - Random?

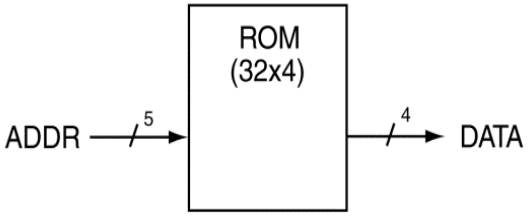
RAM vs. ROM

- Random: refers to the time required to access data
  - RAM and ROM are random access
  - Hard drives, tape drives are not random access

- Functional differences between RAM and ROM
  - RAM = store and retrieve (read & write)
  - ROM = retrieve only (read only)
  - RAM = volatile (data lost on power down)
  - ROM = non-volatile (data not lost on power-down)

- **The final word:**
  - RAMs: Readable & Writeable, Volatile
  - ROMs: Read only, Non-volatile

ROM



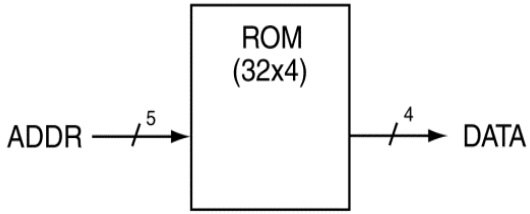

```
module Rom_16x8(input CLK,
                input [3:0] ADDR,
              output logic [7:0] DATA);

    logic [7:0] rom[0:15];

    initial begin
      $readmemh("rom_data.mem", rom, 0,
                                15);
    end

    always_ff @(posedge CLK) begin
        DATA <= rom[ADDR];
    end
endmodule
```

SystemVerilog

```
module rom(input logic [1:0] adr,
           output logic [2:0] dout):
     always_comb
          case(adr)
        2'b00: dout <= 3'b011;
        2'b01: dout <= 3'b110;
        2'b10: dout <= 3'b100;
        2'b11: dout <= 3'b010;
          endcase
endmodule
```

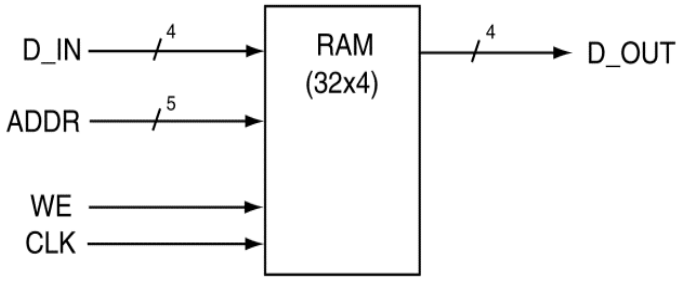
SystemVerilog

VHDL Model for Asynchronous 32x4 ROM




```
entity gen_async_rom is
    Port ( ADDR : in  STD_LOGIC_VECTOR(4 downto 0);
           DATA : out STD_LOGIC_VECTOR(3 downto 0));
end gen_async_rom;

architecture gen_async_rom of gen_async_rom is

   TYPE vector_array is ARRAY(0 to 31) of STD_LOGIC_VECTOR(3 downto 0);

   CONSTANT my_memory: vector_array := ( X"0", X"1", X"2", X"3",
                                         X"4", X"5", X"6", X"7",
                                         X"8", X"9", X"A", X"B",
                                         X"C", X"D", X"E", X"F",
                                         X"F", X"E", X"D", X"C",
                                         X"B", X"A", X"9", X"8",
                                         X"7", X"6", X"5", X"4",
                                         X"3", X"2", X"1", X"F");
begin

   DATA <= my_memory(CONV_INTEGER(ADDR));

end gen_async_rom;
```

RAM



SystemVerilog

```
module ram #(parameter N = 6, M = 32)
            (input  logic         clk,
             input  logic         we,
             input  logic [N-1:0] adr,
             input  logic [M-1:0] din,
             output logic [M-1:0] dout);

  logic [M-1:0] mem [2**N-1:0];

  always_ff @(posedge clk)
    if (we) mem [adr] <= din;

  assign dout = mem[adr];
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD_UNSIGNED.ALL;

entity ram_array is
  generic(N: integer := 6; M: integer := 32);
  port(clk,
       we:  in  STD_LOGIC;
       adr: in  STD_LOGIC_VECTOR(N-1 downto 0);
       din: in  STD_LOGIC_VECTOR(M-1 downto 0);
       dout: out STD_LOGIC_VECTOR(M-1 downto 0));
end;

architecture synth of ram_array is
  type mem_array is array ((2**N-1) downto 0)
       of STD_LOGIC_VECTOR (M-1 downto 0);
  signal mem: mem_array;
begin
  process(clk) begin
    if rising_edge(clk) then
      if we then mem(TO_INTEGER(adr)) <= din;
      end if;
    end if;
  end process;

  dout <= mem(TO_INTEGER(adr));
end;
```